Multi-LLM Chat Client – OpenAI, DeepSeek, Google Gemini, Hugging Face & GroqCloud

Multi-LLM Chat Client is a fully self-contained, locally hosted chat client supporting five AI providers: OpenAI, DeepSeek, Google Gemini, Hugging Face, and GroqCloud. Developed with a focus on **security, simplicity, and professional usability**, the architecture requires no exotic frameworks and relies exclusively on proven technologies: Apache as the web server, Python CGI for server-side logic, and plain HTML/JavaScript/CSS on the client side.

Key highlights: · **Multi-LLM support** – Switch between OpenAI, DeepSeek, Google Gemini, Hugging Face, and GroqCloud via a provider toggle in the LLM Settings panel. Each provider has its own model list, tier selection, and configuration options. · **Clear model labels with release dates** – Model dropdowns show readable labels with the actual release date instead of raw API IDs (e.g. "OpenAI GPT-6 Astra (04.09.2026)" instead of `gpt-6-astra`). Covers all 25 models across all five providers. The underlying value sent to the API is unchanged; only the visible label is friendlier. · **DeepSeek V4.1 Flash** – Since 10 September 2026, [illegible] adds DeepSeek's first native (not bolted-on) image understanding, alongside the existing [illegible] (the old alias [illegible] is retired by DeepSeek, only temporarily routed to V4.1 Flash, and no longer selectable since 12.10.2026) (1M token context windows; legacy [illegible] names retired 24 July 2026 as scheduled; the interim experimental [illegible] was removed on 17 September 2026, fully superseded by [illegible]). · **GPT-6 Astra ready** – OpenAI's new flagship (`gpt-6-astra`, released 3/4 September 2026) joins the existing GPT-5.6 family (Sol, Terra, Luna), GPT-5.5, GPT-4o, and GPT-4.1. Requests use `max_completion_tokens`, the parameter required by all current-generation OpenAI models. · **Real HTTPS via Let's Encrypt** – Since 15 September 2026, the client is reachable via a publicly trusted TLS certificate ([illegible]), issued via `acme.sh` and a DNS-01 challenge against Dynu, replacing the earlier self-signed [illegible] certificate that required manual trust setup on every device. No public port-forwarding is involved — the domain resolves to the server's private VPN address, which stays unreachable from the internet at large. · **Hardened against DNS-over-TLS/HTTPS leakage** and **security-audited** – A structured multi-stage security review (26 July – 15 September 2026) closed stored XSS, path traversal, CORS wildcard, and request-size gaps across the whole CGI backend, added log-line limiting, fixed a Host-header injection risk, and closed a separate DNS-over-TLS bypass in the surrounding VPN gateway. See Security Hardening below for the full account, including what was deliberately left as is and why. · **Responsive design** – The interface adapts to phone and tablet screens (viewport meta tag plus targeted media queries for touch-friendly button sizing and reduced padding on narrow screens), on top of the already-fluid flexbox layout. · **Download Mode** – For very long AI responses, an optional toggle suppresses the live token-by-token rendering in the chat and shows only a running character count instead, then offers the existing export buttons once the response is complete — noticeably faster for multi-hundred-thousand-character answers on modest hardware. · **Working vision pipeline** – Image upload and clipboard paste are fully wired end-to-end for Google Gemini, OpenAI, and DeepSeek ([illegible]): images are base64-encoded client-side and delivered as native `inline_data` (Gemini) or `image_url` (OpenAI/DeepSeek, OpenAI-compatible format) content blocks. Model capability detection ([illegible]) is populated per provider instead of defaulting to "no image support". · **Multi-file upload** – Select and send multiple files simultaneously. Contents are combined and sent as context with per-file headers and separators. · **Audio recording via microphone** – Record audio directly in the browser and send it to the AI. Supported natively by Google Gemini ([illegible]). OpenAI currently has no audio-capable model in this client. The Record Audio button appears automatically only when an audio-capable model is active. · **Unique context management** – Delete any individual message along with all subsequent ones. The chat remains consistent and token usage is dynamically recalculated. · **Maximum security** – API keys are never visible on the client side, uploads are protected against executable files via magic-byte inspection, and sessions are stored with restrictive file permissions. · **No exotic frameworks** – Everything is based on Apache, Python 3, Bash, and plain HTML/JavaScript/CSS. No Node.js, no React, no build pipeline. · **Professional export functions** – PDF, Markdown, TXT, and RTF for the entire chat or individual messages, plus direct copy to clipboard (client-side, no server roundtrip). · **Multi-language support** – Full UI translation via external `language.xml` (English, German, Spanish, extensible with a custom language slot). · **Kompressor (context compression)** – Automatic intelligent compression of the chat history when the context window fills up. A second LLM call summarizes the oldest 50% of messages and injects the summary into the system prompt — enabling indefinitely long conversations without losing context. · **Quota & limit banners** – Persistent visual banners for exhausted credit (red, paid providers) and daily limits (blue, free-tier providers), each with a close button. · **Context window exceeded handling** – When the maximum context size is reached, an interactive in-chat box offers two choices: continue with compressed context carried forward, or start a clean new chat. The current session is saved automatically in both cases. · **Clipboard integration** – Ctrl+V handler with dialog for text, images, and protection against accidentally pasting file paths. · **Streaming responses** – AI answers appear token by token, just like ChatGPT or Claude. · **429 Rate-limit handling** – Automatic retry with countdown display for Google Gemini Free Tier limits. · **Transparent error diagnostics** – API error responses now surface the actual provider-side error message (instead of a blank string) whenever the error doesn't match a known quota/context pattern. · **Deploy verification** – `deploy.sh` prints MD5 checksums of every file copied into production, enabling immediate cross-checking against the source repo without a separate manual step. · **Included tooling** – The [illegible] script exports the entire repository as a single text file, ideal for working with AI assistants.

Table of Contents

- Overview
- Architecture
- Unique Context Management
- Features in Detail
- Chat Interface
- Streaming Responses
- Clipboard Handler (Ctrl+V)
- File Upload with Security Check
- Upload Placeholder System
- DeepThink Mode
- Model Detection & Capabilities
- Vision (Image) Support
- Multi-Language System
- Settings (Toggles instead of Radio Buttons)
- Session Management
- Export Functions
- Feedback Buttons & Logging
- Dynamic Context Display
- File Card Display
- Audio Recording
- Kompressor — Intelligent Context Compression
- Quota & Limit Banners
- Context Window Exceeded Handling
- Download Mode
- Responsive Design
- DeepSeek V4 Migration
- 19 July 2026 Maintenance & Feature Update
- 28 August 2026 Fix: Explicit DeepSeek Thinking-Mode Control
- Security Hardening (26 July – 15 September 2026)
- HTTPS Migration: From Self-Signed to Let's Encrypt
- 17 September 2026: Model Display Labels & Cleanup
- The Helper Script repo2txt.sh
- Security Architecture in Detail
- Deployment & Usage
- Prerequisites
- Installation
- Configuration
- Deploy Scripts
- Project Structure
- Model Configuration
- Design Manifest
- Known Limitations & Technical Notes
- Dependencies
- Conclusion / Why This Project Stands Out

Overview

Multi-LLM Chat Client is a **local web application** that communicates with external AI APIs exclusively via server-side Python CGI proxy scripts. Developed for a private Debian server environment, it can run on any Linux system with Apache 2.4 and Python 3. The goal was a **secure, extensible, and user-friendly** chat client without cloud dependencies and with full control over data and API credentials.

The project has grown continuously over several weeks of active development, accumulating features like streaming responses, session management, export functions, multilingual support, clipboard integration, intelligent context compression, audio recording, and robust security measures — without ever introducing external JavaScript frameworks or a build toolchain.

The entire client logic resides in a single `index.html` file (~9,000 lines). All UI texts are externalized to `language.xml`. All server-side operations are handled by 15 Python CGI scripts in `/cgi-bin/`.

Architecture

The architecture is intentionally simple but well thought out:

1. Client Layer

- Pure HTML5/JavaScript/CSS3, served via Apache.
- No build tools, no Node.js, no external JavaScript libraries (except PDF.js 3.11.174, loaded via CDN, for in-browser PDF text extraction).
- The entire client logic — message processing, UI updates, streaming reception, language switching, clipboard handling, session management, context estimation — is encapsulated in a single `index.html`.
- All UI texts are loaded from an external `language.xml` at startup via `fetch()`. No hardcoded UI strings exist in the HTML.
- Settings are persisted in `localStorage` with version-controlled schema migration.

2. Server Layer

- **Apache 2.4** with `mod_cgi` enabled. HTTPS enforced via SSL configuration.
- **Python 3 CGI scripts** under `/cgi-bin/` handle all server-side operations:

Script	Function
`openai_api.py`	Streaming proxy to OpenAI Chat Completions endpoint (native format)
`deepseek_api.py`	Streaming proxy to DeepSeek Chat Completions endpoint (OpenAI-compatible)
`google_api.py`	Proxy to Google Gemini API with format conversion (OpenAI ↔ Gemini)
`hugging_api.py`	Streaming proxy to Hugging Face Inference router (OpenAI-compatible)
`groq_api.py`	Streaming proxy to GroqCloud API (OpenAI-compatible, LPU hardware)
`compress_context.py`	Context compression — summarizes oldest 50% of messages via second LLM call
`deepseek_models.py`	Queries DeepSeek `/v1/models` endpoint at startup for live model detection
`save_session.py`	POST: receives `{sessionId, messages}`, validates ID, writes JSON to disk
`load_session.py`	GET: returns session list with previews; POST `{sessionId}`: returns full session
`delete_session.py`	POST `{sessionId}`: removes session JSON file
`export_pdf.py`	Server-side PDF export via ReportLab
`export_markdown.py`	Server-side Markdown export
`export_txt.py`	Server-side TXT export
`export_rtf.py`	Server-side RTF export (no external library, manual RTF encoding)
`feedback_log.py`	Writes Like/Dislike feedback entries to the server log
`get_log.py`	Reads and returns the server log file content

- **API keys** are provided exclusively via Apache environment variables in `/etc/apache2/envvars` — `OPENAI_API_KEY`, `DEEPSEEK_API_KEY`, `GOOGLE_API_KEY`, `HF_API_KEY`, `GROQ_API_KEY`. They are **never** present in client code or HTTP responses.
- A single `ScriptAlias /cgi-bin/ /var/www/deepseek-chat/cgi-bin/` covers all scripts — no Apache configuration changes are needed when adding new scripts.

3. Data Storage

Location	Content	Permissions
`/var/www/deepseek-chat/sessions/`	Chat session JSON files	`chmod 700` (dir), `chmod 600` (files)
`/var/www/deepseek-chat/logs/multi_llm_chat.log`	Server activity log (no API keys, no session content)	`www-data` owned
`/var/www/deepseek-chat/compressions/`	Compression result files (one per compression round)	`www-data` owned

Location	Content	Permissions
Browser [illegible]	User settings (with version migration), language, model preferences	Client-side only
[illegible]	All UI texts for all languages	Loaded via [illegible] at page load

4. Helper Scripts

- [illegible] — copies source repo to production directory, sets correct ownership/permissions, reloads Apache.
- [illegible] — copies changed files from production back to the source repo.
- [illegible] — installs [illegible] and [illegible] in the production directory.
- [illegible] — creates a Git tag with auto-incremented version (e.g. [illegible]) and pushes it. Runs [illegible] automatically to avoid conflicts with existing remote tags.
- [illegible] — tests the provider APIs with the same parameters as the CGI proxies (model list, text at the full output limit, image, audio, DeepSeek DeepThink). Reads the model lists and limits from [illegible], API keys from [illegible]; run as root on the server. Host guard (since 14.10.2026): before any request the script aborts with exit code 2 if Apache2 is not installed, the production directory [illegible] is missing, [illegible] is not readable, or the API keys of [illegible]
- [illegible] — automatic browser test of the web page (Playwright/Chromium, one-time setup in the script header). Provider answers and the session/bookmark scripts are simulated, so it costs nothing; the export scripts run for real. Checks: loading, language (every visible text must belong to the chosen language, in all dialogs), settings and model lists, chat for all five providers, error cases (credit banner, retry, overloaded, context too large), exports in all languages (title, roles, accents, [illegible]
- [illegible] — exports the entire repository as a single delimited text file for AI assistants.

Unique Context Management

One of the standout features is the ability to **delete any individual message along with all subsequent ones**. This goes far beyond the typical "delete last message" and allows flexible correction of conversation history at any point.

Implementation: - Each message (user & AI) receives a unique ID (format [illegible]) and is stored in the array [illegible]. - [illegible] determines the index of the target message, truncates the array from that index onwards, and removes all following DOM elements (messages + dividers). - [illegible] immediately recalculates the estimated token count and the percentage context utilization [illegible]

Why this is unique: Most chat clients only allow deletion of the last message or no history manipulation at all. Here, the user can **define any point in the conversation as a new starting point** — ideal for testing prompt variations, correcting mistakes mid-conversation, or cleaning up the context window without discarding the entire chat.

Regenerate function: Every AI response includes a "Regenerate" button that removes the current response from both context and DOM, then issues a new API call based on the same user message and full preceding history.

Features in Detail

Chat Interface

- **Fixed Dark Mode** — Background [illegible], text [illegible], accent [illegible]. No light mode option by design.
- **Server header** (3 lines): server name (blue [illegible]), internal IP address, dynamic context utilization with active model name, detected model IDs from the DeepSeek API.
- **Message containers:** hover-triggered action buttons (feedback, per-message export, delete). User messages appear in blue ([illegible]), AI responses in white on dark.
- **Textarea:** expands on focus from 40px to 120px via CSS transition. Enter sends the message; Shift+Enter inserts a line break.
- **Strict pill-style design:** border-radius 20px, height 38px for all buttons — no square buttons anywhere in the UI.
- [illegible] on all message content preserves formatting from AI responses.
- Auto-scroll to the latest message is active during and after streaming.

Streaming Responses

All AI responses are received and displayed **token by token** using Server-Sent Events (SSE):

- All five CGI proxy scripts send their respective API requests with [illegible] (or equivalent) and forward the raw SSE event stream directly to the client without buffering.
- [illegible] reads the stream via the [illegible] API with [illegible].
- Each received chunk is appended to the active message DOM element in real time.
- **Technical SSE headers** set by all CGI proxy scripts: [illegible]
- The psychological effect is significant: first tokens appear within ~300ms instead of waiting 5-10 seconds for a complete response.
- Both [illegible] and [illegible] use identical streaming logic.

OpenAI Integration

- **Endpoint:** [illegible]
- **Architecture:** Native OpenAI Chat Completions format — no format conversion required. SSE stream forwarded directly by [illegible].
- **API key:** [illegible] via Apache environment variables.
- **Free Tier models:** [illegible]
- **Paid Tier models:** [illegible]
- **Output token parameter:** [illegible] — required by all current OpenAI models (GPT-4o/4.1 accept it too, so a single parameter works across the entire lineup). The older [illegible] parameter is rejected by GPT-5.x models with HTTP 400 ([illegible]).
- **Image input:** [illegible], and the entire GPT-5.6 family accept image input. Images are sent as [illegible] content blocks with a base64 data URL ([illegible]).
- **Audio input:** Not available in this client (changed 12.10.2026). Live tests with [illegible] showed that [illegible] and [illegible] reject [illegible] over Chat Completions (HTTP 400: "Content blocks are expected to be either text or image_url type"), so both were removed from [illegible] and the Record Audio button is hidden for OpenAI. OpenAI's dedicated [illegible]
- **No free API tier for GPT-5.x:** OpenAI does not offer a genuinely free tier for GPT-5.6/5.5/5.6 in the API — the "Free" grouping in this client denotes the cheapest available models ([illegible]), not a $0 quota.
- **No retirement of the aliases (12.10.2026):** The earlier notice of an API-wide shutdown of [illegible] and [illegible] on 23 October 2026 was incorrect. On that date OpenAI retires only specific snapshots ([illegible] and others); the aliases [illegible] and [illegible] are not on the deprecation list. Please re-test [illegible]
- **GPT-6.1 Sol / GPT-6 Luna (12.10.2026):** [illegible] (29.09.2026, [illegible] per 1M tokens) and [illegible] (22.09.2026, [illegible]) added — text + image, 1,050,000 context / 128,000 output. [illegible] (superseded by 6.1) was deliberately not added.
- **Output limit corrected (12.10.2026):** [illegible] has 128,000 output tokens (was wrongly 16,384).
- **Error handling (12.10.2026):** [illegible] now classifies OpenAI's error codes (see "Error codes" in the OpenAI docs): HTTP 503 [illegible] and HTTP 429 [illegible] / rate limit are temporary ([illegible]) — the client retries up to 3 times with a 15-second countdown and then shows a plain message; HTTP 429 [illegible]
- The DeepThink button and indicator are hidden when OpenAI is the active provider.
- System prompt identifies the active model: *"You are [model], an AI assistant made by OpenAI."*

Google Gemini Integration

- **Endpoint:** [illegible]
- **Architecture:** [illegible] converts the OpenAI-compatible internal message format to Gemini's [illegible] format, sends the request, and converts the Gemini SSE response back to the OpenAI SSE format expected by the client.
- **API key:** [illegible] via Apache environment variables.
- **Free Tier models:** [illegible] (rate limits per project, see AI Studio)
- **Paid Tier models:** [illegible]
- **Image input:** All Gemini models accept image input, sent as [illegible] blocks in Gemini's native format (same mechanism used for audio).
- **Audio input:** All Gemini models support audio natively. Audio is sent as [illegible] blocks in Gemini format. The recording button is always visible when Google Gemini is active.
- **Retired models:** [illegible] (shut down 1 June 2026) and [illegible] (retired earlier) have been removed from all model lists, [illegible], and [illegible]. The default fallback model was updated from [illegible] to [illegible].
- **Availability of the 2.5 models (12.10.2026):** The earlier notice of a shutdown on 16 October 2026 was incorrect (third-party source, referred to Vertex AI). Google lists no shutdown date for [illegible] / [illegible] ("not deprecated, served until further notice") but limits access to users who have used them before.
- **Gemini 3.x (12.10.2026):** [illegible] (GA 02.09.2026) and [illegible] (GA 21.07.2026) added — multimodal (text, image, audio, video, PDF), 1,048,576 tokens input / 65,536 output. Thinking is on by default and thinking tokens count towards [illegible] (deprecated since 08.10.2026, auto-routed by Google to [illegible]
- **Output limit corrected (12.10.2026):** [illegible] has 65,536 output tokens (was wrongly 8,192, the Gemini 2.0 Flash value).
- **[illegible] replaced (12.10.2026):** With the current key [illegible] got HTTP 404 "no longer available to new users" for [illegible]; Google names [illegible] as the replacement. It now sits in [illegible] (1,048,576 input / 65,536 output; text, image, audio, video, PDF; thinking on; label date only "02.2026" because no exact release date [illegible]
- The DeepThink button and indicator are hidden when Google Gemini is the active provider.

Hugging Face Integration

- **Endpoint:** [illegible] — the Hugging Face inference router selects the fastest available provider automatically.
- **Architecture:** OpenAI-compatible format — no conversion required. SSE forwarded directly by [illegible].
- **API key:** [illegible] — a Write token from [illegible] with "Make calls to Inference Providers" permission.
- **Free Tier models:** [illegible]
- **Paid Tier models:** [illegible]
- **Removed:** [illegible] — no longer deployed by any Inference Provider on the Hugging Face router as of 19 July 2026.
- **Model list replaced (12.10.2026):** Checked against the router's model list ([illegible]): [illegible] (not a chat model), [illegible] (supported by no enabled provider) and [illegible] (no longer exist) were removed; [illegible]
- The DeepThink button and indicator are hidden when Hugging Face is active.

GroqCloud Integration

- **Endpoint:** [illegible]
- **Architecture:** OpenAI-compatible format — no conversion required. SSE forwarded directly by [illegible].
- **API key:** [illegible] via Apache environment variables.
- **Important:** A [illegible] header is required in all requests — without it, Cloudflare returns error code 1010 and blocks the request.
- **Free & Paid Tier models:** [illegible] (default), [illegible] (lineup replaced on 12.10.2026, see below).
- All models run on GroqCloud's LPU (Language Processing Unit) hardware, delivering very low inference latency.
- **Documentation cleanup:** The [illegible] header previously documented [illegible] (deprecated by Groq since 20 March 2026) and [illegible] (deprecated since 8 October 2026) as supported models — both were already unreachable and have been removed from the header. The actual model arrays in [illegible] were already correct; only the [illegible]
- **Lineup replaced (12.10.2026):** Groq's model list ([illegible]) no longer contains any of the earlier five models ([illegible]; HTTP 404). New: [illegible] and [illegible] (131,072 context, frontend output limit 32,768 of the [illegible]
- The DeepThink button and indicator are hidden when GroqCloud is the active provider.

LLM Settings Panel

A dedicated **LLM Settings** panel (separate from the main Settings panel) keeps all provider-specific configuration out of the main interface.

- **Provider selection:** Toggle between OpenAI, DeepSeek, Google Gemini, Hugging Face, and GroqCloud — exactly one provider active at a time.
- **OpenAI options:** Free / Paid plan toggle with automatic model list update.
- **DeepSeek options:** Default mode (Normal Chat / DeepThink), Privacy toggle ([illegible] header).
- **Google options:** Free / Paid plan toggle with automatic model list update.
- **Hugging Face options:** Free / Paid plan toggle with automatic model list update.
- **GroqCloud options:** Free / Paid plan toggle with automatic model list update.
- **Kompressor options:** Enable/disable toggle, compression provider selection (paid providers only), compression model selection. Default: enabled, DeepSeek / [illegible].
- **Model dropdown:** Always visible, content updates automatically based on the active provider and selected plan.
- All LLM settings are persisted in [illegible] and survive page reloads.

429 Rate Limit Handling

The Google Gemini Free Tier enforces strict rate limits (5 RPM, 20 RPD). The client handles these gracefully without showing a raw error:

- On HTTP 429, the client automatically retries up to **3 times** with **15-second intervals**.
- During the wait, a countdown is displayed directly in the chat: *"Rate limit reached – waiting 15 seconds and retrying... (Attempt 1/3)"*.
- After 3 failed attempts, the daily limit check triggers the blue limit banner if applicable.
- The retry logic distinguishes between temporary RPM limits (retryable) and exhausted daily quota (non-retryable).
- Verbose error details are written to the server log for diagnosis.

Clipboard Handler (Ctrl+V)

A sophisticated clipboard handler intercepts all paste events and responds intelligently based on content type:

Text content → A paste dialog appears with two options: - *"Insert at cursor position"* — inserts the text directly into the input field at the current cursor position. - *"Attach as file"* — treats the clipboard text as `clipboard.txt` and attaches it as a file card to the next message.

Image content → A thumbnail preview box appears above the input field showing the image, its dimensions in KB, and a remove button. The image is ready to be sent with the next message if the active model supports images.

File paths from file managers (XFCE/Thunar, KDE/Dolphin, etc.) → Blocked with an alert:

> *"Files copied in the file manager cannot be read by the browser. Please use the Upload button instead."*

Technical background: On Linux/X11/Firefox, `e.preventDefault()` does not reliably block paste events for content originating from file managers. The implemented solution allows the paste, then immediately checks the input field content via `setTimeout(0)` and clears it if file paths are detected. Detection logic: 2 or more lines where every non-empty line begins with `/` or `file://`. A `requestAnimationFrame` call ensures the input field is visually cleared before the alert dialog appears.

File Upload with Security Check

- **Accepted formats**: `.txt`, `.pdf`, `.doc`, `.docx`, `.jpg`, `.jpeg`, `.png`, `.csv`, `.xlsx`, `.pptx`
- **Content-extractable formats** (text sent to the AI): `.txt`, `.pdf`
- **Other accepted formats**: attached as binary context (without text extraction)
- **Maximum file size**: 10 MB per file
- **Maximum extracted content**: dynamic — calculated as 75% of the active model's context window in characters: [illegible]

Dynamic upload limit examples:

Model	maxContextTokens	Max file content
`deepseek-flash`	1,048,576	~3,145,000 chars
`deepseek-v4-pro`	1,048,576	~3,145,000 chars
`gemini-2.5-flash`	1,048,576	~3,145,000 chars
`gemini-3.0-flash`	1,048,576	~3,145,000 chars
`gemini-3.0-flash-lite`	1,048,576	~3,145,000 chars
`gpt-5.4-[illegible] / [illegible] / [illegible]`	1,048,576	~3,145,000 chars
`gpt-4o / gpt-4o-mini`	128,000	~384,000 chars

Magic byte inspection (first 20 bytes) detects and blocks executable files regardless of filename extension:

Platform	Format	Hex Signature
Windows 32/64 bit	PE/NE Executable	`4D 5A`
Linux 32 bit	ELF32	`7F 45 4C 46 01`
Linux 64 bit	ELF64	`7F 45 4C 46 02`
ARM 32 bit	ELF32 ARM	`7F 45 4C 46 01 01 01 00 … 02 00 28 00`
ARM 64 bit	ELF64 AArch64	`7F 45 4C 46 02 01 01 00 … 02 00 B7 00`
macOS 32 bit	Mach-O	`CE FA ED FE`
macOS 64 bit	Mach-O	`CF FA ED FE`
macOS Universal	Fat Binary	`CA FE BA BE`
macOS/iOS ARM 32	Big Endian	`FE ED FA CE`
macOS/iOS ARM 64	Big Endian	`FE ED FA CF`
Linux/macOS	Shell Script	`23 21` (#!)
Python	Bytecode (.pyc)	`55 0D 0D 0A`

PDF extraction: Uses PDF.js 3.11.174 loaded from CDN with automatic fallback to a secondary CDN. Extraction progress is displayed page by page. Extraction timeout: 30 seconds.

Pre-upload context check: Before extracting file content, the client estimates whether adding the file would exceed the dynamic upload limit. If it would, the upload is blocked with a clear error message before any content is extracted.

Umlaut Placeholder System

A unique solution for a fundamental problem with the DeepSeek API and German text:

Problem: DeepSeek internally replaces German umlauts in file content with ASCII equivalents (e.g. [illegible]). This behavior cannot be suppressed via system prompts or API parameters.

Solution: Before sending file content to DeepSeek, umlauts are replaced with unique bracketed placeholders. DeepSeek returns these placeholders unchanged. JavaScript replaces them back to real umlauts after receiving the response.

Original	Placeholder
ä	`[[ae]]`
ö	`[[oe]]`
ü	`[[ue]]`
ß	`[[ss]]`
Ä	`[[AE]]`
Ö	`[[OE]]`
Ü	`[[UE]]`

Critical implementation detail: Both `[illegible]` and `[illegible]` use exclusively **Unicode escape sequences** (`[illegible]` instead of ä) and `split().join()` instead of regex — essential to prevent corruption when files are transferred via Git or edited in text editors.

The decode runs **both during streaming** (token by token) and again after the complete response is received, ensuring no placeholders remain visible even with partial chunk delivery.

This system is applied **only to file content**, never to regular user messages or system prompts.

DeepThink Mode

DeepThink is a dedicated mode for deep analytical reasoning, exclusively available when DeepSeek is the active provider:

- Activated via a dedicated pill-style button in the second button row below the input field.
- When active, `deepseek-flash` (or `deepseek-v4-pro`, if selected in the model dropdown) is used with DeepSeek's Thinking Mode explicitly enabled at high reasoning effort. The more capable `deepseek-v4-pro` is selected manually via the model dropdown for maximum reasoning depth in either mode. **Note:** prior to 19 July 20[illegible]
- The button changes visually: inactive (dark `[illegible]`) → active blue (`[illegible]` background, `[illegible]` border and text).
- An indicator bar appears below the button row: *"DeepThink Mode active: In-depth analysis in progress"*.
- Context limits and output token limits are automatically adjusted based on the active model's `MODEL_CONFIG` entry.
- The mode is recorded with each message (field `[illegible]`) and shown in all export formats.
- The default mode (Chat or DeepThink) can be configured in Settings and is persisted in `localStorage`.
- The DeepThink button and indicator are automatically hidden when any non-DeepSeek provider is active.

Model Detection & Capabilities

At startup, `index.html` queries `/api/[illegible]`, which calls the DeepSeek `/v1/models` endpoint live:

- The returned model IDs are stored in `[illegible]` and used internally for capability checks (see below). They are **not** used to render the header — the server header (`[illegible]`) always shows the currently **selected** model (`[illegible]`), matching the behavior across all five providers. Earlier versions incorrectly displayed the full `[illegible]` list [illegible]
- A `MODEL_CAPABILITIES` map defines which models support which input types, populated per provider based on each backend's documented capabilities: [illegible]
- Since 17 September 2026, a separate `MODEL_DISPLAY_LABELS` map (with a `[illegible]` helper) decouples the dropdown's visible text from the API value actually sent — see Model Display Labels & Cleanup for the full story.
- `currentModelSupportsImages()` checks `[illegible]` (falling back to the current `[illegible]` dropdown value) against `MODEL_CAPABILITIES`. Earlier versions checked `[illegible]` instead — a DeepSeek-only array that was never populated for other providers — which meant image upload and paste were silently blocked for **every** provider and model, including [illegible]
- If an image is pasted via clipboard or a `.jpg`/`.png` file is uploaded, and the current model does not support images, the operation is blocked with an alert before any upload occurs.
- This architecture is **forward compatible**: adding image support for a model only requires adding or updating its entry in `MODEL_CAPABILITIES` — but note that `MODEL_CONFIG` (see Model Configuration) also needs a matching entry for the model to get correct context/output limits rather than silently falling back to DeepSeek's defaults.

Vision (Image) Support

Image upload and clipboard paste are fully wired end-to-end for Google Gemini and OpenAI. This was a significant gap closed on 19 July 2026 — previously, images were accepted by the UI but never actually reached any model.

Client side (`index.html`) – **File upload**: When an image file (`.jpg`, `.jpeg`, `.png`, etc.) is selected and the active model supports images (per `currentModelSupportsImages()`), the file is read via `FileReader.readAsDataURL()`, and the base64 payload (with the `data:...;base64,` prefix stripped) is stored in `[illegible]`, with the MIME type in `[illegible]`. · **Clipboard paste**: Pasting an image (Ctrl+V) performs the same base64 [illegible]

Server side: · `google_api.py`: `[illegible]` accepts `image_data`/`image_mime_type` and appends the image as an `inline_data` part to the last user message — the same mechanism already used for audio. · `openai_api.py`: The last user message's `content` is built as a list combining the text part with an optional `input_audio` block and an optional `image_url` block [illegible]

Verification: Tested live with `gemini-2.5-flash` and `gpt-4o-mini` — both correctly described the content of an uploaded screenshot in detail.

Multi-Language System

The UI supports multiple languages loaded from an external `language.xml` file. No UI strings are hardcoded in `[illegible]`.

Currently included languages: - English (`en`) — default, no address form distinction - German (`de`) — with formal/informal address form (Sie/Du) - Spanish (`es`) — with formal/informal address form (Usted/Tú) - Custom slot (`custom`) — activated by setting `[illegible]` in `language.xml`

Technical implementation: - All UI texts are referenced by numeric IDs. `t(100)` returns the Send button label in the current language. - `loadLanguage()` fetches and parses `language.xml` via `fetch()` at page load. - `t(id)` — returns text for the current language, falls back to English if the ID is not found. - `t(id, ...args)` — supports `{0}`, `{1}`, … placeholder substitution. - `[illegible]` — returns the appropriate text based on the selected address form. - Language switching is instant, no page reload required. - The selected language is persisted in `localStorage`.

Address form system (German/Spanish): - Languages declare `has_address_form="true"` in `language.xml`. - For such languages, the Settings panel shows an "Address Form" group (Formal/Informal). - The selected form affects: system prompt (enforces consistent AI response style), input placeholder text, all settings description texts.

System prompt is built dynamically per request from: 1. Base prompt (text IDs 29/30 for formal/informal) 2. DeepThink addition (text IDs 31/32) 3. A strict file-handling instruction always appended in English regardless of UI language — ensuring consistent AI behavior when processing file content.

Settings (Toggles instead of Radio Buttons)

All settings use **toggle switches** (sliding left-to-right), never radio buttons or checkboxes.

Group	Setting	Toggle Color
Language	EN / DE / ES / Custom	Green
Address Form	Formal / Informal	Green
Default Mode	Normal Chat / DeepThink	Blue
Privacy	Do not use data for training	Green

Toggle behavior: - Within a group, toggles behave as radio buttons: activating one deactivates all others in the group. - Clicking anywhere on the `setting-row` row activates that toggle — not just the toggle element itself. - Active items receive a colored background: `[illegible]` (green groups) or `[illegible]` (blue groups).

Privacy toggle: Sets the HTTP header `X-No-Training: true` in all API requests to DeepSeek, utilizing DeepSeek's opt-out mechanism for training data.

Settings persistence: All settings are stored in `localStorage` under the key `[illegible]`. Current schema version: `[illegible]`. The `migrateSettings()` function ensures backward compatibility — missing fields are filled with defaults, unknown modes are normalized.

Session Management

Every conversation is automatically managed as a server-side session:

- **Session ID format**: `[illegible]` (e.g. `[illegible]`) — generated client-side, validated server-side via regex before any file I/O.
- **Auto-save**: After every sent message pair (user + AI), the complete `conversationHistory.messages[]` array is serialized and saved to the server as a JSON file.
- **Session file**: `{sessionId}.json` in `[illegible]`, chmod 600, owned by `www-data`.
- **Load Chat History modal**: Lists all saved sessions with ID, date, message preview (first 80 chars), and message count. Each session has [Load] (green) and [Delete] (red) buttons.
- **Load behavior**: The current chat is auto-saved first, then the selected session is restored — full message history, UI reconstruction, context estimation recalculation.
- **Delete**: The JSON file is removed from the server immediately without confirmation dialog.

CGI endpoint details: - `save_session.py` — POST receives `{sessionId, messages}`, validates ID format (regex), writes `sessions/{sessionId}.json` - `load_session.py` — GET returns `[{id, preview, count, date}]`; POST `{sessionId}` returns full `{messages: [...]}` - `delete_session.py` — POST `{sessionId}` removes `sessions/{sessionId}.json`

Export Functions

Global export (dropdown button in the main button row):

Format	Generation	Notes
PDF	Server-side (`export_pdf.py`, ReportLab)	Header, statistics, table of contents, full chat
Markdown	Server-side (`export_markdown.py`)	Identical structure to PDF, with anchors
TXT	Server-side (`export_txt.py`)	Plain text with separators
RTF	Server-side (`export_rtf.py`)	Manual RTF encoding, all non-ASCII characters as RTF escape codes
Copy to clipboard	**Client-side only (no server roundtrip)**	Plain text assembled in JavaScript, written via `navigator.clipboard.writeText()`

Per-message export (hover button on each message):

Format	Generation
TXT	Client-side (JavaScript Blob, `URL.createObjectURL()`)
Markdown	Client-side
RTF	Client-side
PDF	Server-side (single message sent to `export_pdf.py`)

Export language: All captions in the exports (title, "Date", "Time", "Messages", role names, statistics headings, table of contents, footer) follow the UI language. The frontend sends them with each export request as `[illegible]` (built by `[illegible]`) from `language.xml`, IDs 283–307; the four CGI scripts use German defaults if no labels arrive. RTF encodes all non-ASCII characters (umlauts, Spanish accents, ¿, ¡, …) as RTF escape codes. The sending banner, the microphone messages and the close tooltips also come from `language.xml` (IDs 279–282), and number formats follow the UI language.

Export content (PDF / Markdown global export): - Header: server name, IP, export date, active language, address form - Statistics: total message count, modes used (chat/deepthink), files attached, estimated token count, session duration - Table of contents with all message timestamps - Full chat history with per-message timestamps and mode indicators

Copy to clipboard: Assembled client-side in TXT format and written via `navigator.clipboard.writeText()`. A 2-second "Copied" confirmation replaces the Export button label, then resets automatically.

PDF technical note: Binary PDF data is written exclusively via `sys.stdout.buffer` with HTTP headers encoded as bytes — avoiding the "bad header" error that occurs when mixing `print()` (text mode stdout) with binary content.

Feedback Buttons & Logging

Four buttons appear on hover for each AI response (left side, bottom row):

- **Copy** — Copies message text to clipboard; shows "Copied" for 2 seconds, then resets.
- **Like** — Marks the response positively (blue highlight); sends a `LIKE` entry to the server log. Clicking again removes the like.
- **Dislike** — Marks the response negatively (red highlight); sends a `DISLIKE` entry. Like and Dislike are mutually exclusive.
- **Regenerate** — Removes the current AI response from context array and DOM, then calls the API again with the same user message and full preceding history.

Server log format (`[illegible]`):

```
[illegible] | IP: [illegible] | POST /cgi-bin/deepseek-api.py | Status: 200
[illegible] | IP: [illegible] | FEEDBACK | LIKE | msg_5 | "First 60 chars of response..."
```

Never logged: API keys, full session contents, or message text beyond the 60-character feedback preview.

Dynamic Context Display

The server header shows four lines of real-time information: 1. Server name (blue accent) 2. `[illegible]` 3. `[illegible]` 4. `[illegible]` (live from `[illegible]`)

Context utilization calculation: - Estimated tokens = sum of character counts in the last `[illegible]` messages × `[illegible]` (0.25) - System prompt tokens are added as a fixed overhead - Percentage = estimated tokens / `[illegible]` × 100

Warning system: Above 90% context utilization, the context line turns red and blinks (CSS keyframe animation, opacity 0 → 1, 1-second cycle). This provides a highly visible early warning.

The display updates automatically after every sent message, every deleted message, and every model switch.

File Card Display

When a file is uploaded or clipboard text is attached, the user message displays a compact **file card**:

```
[illegible]
│ [PDF] │ filename.pdf │
│ [illegible] │ PDF Document │
[illegible]
```

- File type badge derived from the file extension (PDF, TXT, XLSX, DOCX, etc.)
- Filename truncated to 30 characters with `...` if longer
- Audio recordings display an `AUDIO` badge with the localized label
- Multi-file uploads generate one card per file; all filenames appear in the info bar separated by `;`

Audio Recording

The client includes a built-in **microphone recording button** enabling direct voice input to audio-capable models.

- **Visibility**: Controlled by `updateAudioButtonVisibility()`, called on every model change. Visible only when the active model is listed in `AUDIO_CAPABLE_MODELS`.
- **Audio-capable models** (`AUDIO_CAPABLE_MODELS` constant):
- Google Gemini: `gemini-2.5-flash`, `gemini-2.5-pro-preview`, `gemini-2.0-flash`, `gemini-2.0-flash-lite`
- OpenAI: none (since 12.10.2026 — `gpt-4o` and `gpt-4.1` reject `input_audio`, HTTP 400)
- **Recording flow**: `getUserMedia()` → `MediaRecorder` API → chunked recording (10ms intervals) → Blob assembled on stop → base64-encoded.
- **MIME type auto-detection**: `audio/webm` (Chrome/Firefox) or `audio/mp4` (Safari) — detected at runtime via `MediaRecorder.isTypeSupported()`.
- **After recording**: Audio data is shown in the `fileInfo` box as an AUDIO badge card.
- **Request payload**: `audio_data` (base64 string) and `audio_mime_type` are added to the JSON body alongside the text message. The `has_file` flag is **not** set — no file-processing system prompt is injected for audio.
- **Mutual exclusivity**: File upload and audio recording are mutually exclusive. Starting a recording clears any pending file attachment and vice versa.
- **Backend — Google** (`google-api.py`): Audio is appended to the last user message as an `inline_data` block in Gemini's native format.
- **Backend — OpenAI** (`openai-api.py`): Audio is appended as an `input_audio` block with `format: wav or mp3` (currently unused: no OpenAI model is audio-capable).
- **Maintenance rule** (Manifest rule 8.1): Whenever an integrated provider adds or removes audio support for a model, `AUDIO_CAPABLE_MODELS` in `index.html` **must** be updated immediately.

Kompressor — Intelligent Context Compression

Every language model has a finite context window. In long sessions — particularly with large file uploads, multi-hour conversations, or extensive analysis workflows — the context window eventually fills, causing API errors (HTTP 400/413) that force the user to start over and lose the entire conversation thread.

The **Kompressor** solves this problem automatically and transparently, without any user action required.

Core Concept

Instead of truncating old messages or forcing a manual restart, the Kompressor **summarizes** the oldest half of the conversation via a dedicated second LLM call. This summary is injected into the system prompt of all subsequent requests. The active model effectively "remembers" the summarized past — the conversation can continue indefinitely.

Activation Thresholds

Threshold	Action
70% context utilization	First compression round
85% context utilization	Second compression round
95% context utilization	Third compression round

Each threshold fires at most once per session cycle. After manual message deletion, if the context percentage drops below the last triggered threshold, all threshold tracking resets.

Compression Process (Step by Step)

1. After each sent message, `updateContextStatus()` recalculates context utilization.
2. If a threshold is exceeded, `compress_context.py` is called **before** the main API call.
3. The oldest 50% of messages are extracted. The cutoff advances to the next user message — ensuring API compatibility (context must always start with a user turn).
4. Base64 data, images, and multimedia content are filtered out — only plain text is sent to the compression LLM.
5. The compression LLM (configurable provider and model) returns a structured summary.
6. The old messages are replaced by a single compressed entry (flag `compressed: true`).
7. The summary text is prepended to the system prompt for all subsequent API calls — never sent as a standalone `assistant` message (which causes 400 errors).
8. The compressed context is saved to disk. The main API call proceeds with the reduced context.

Smart Summary Discard

When the user manually deletes messages and the context percentage drops below the **last triggered threshold** (not simply below 70%), the compression summary is automatically removed from the system prompt and all threshold counters reset. This ensures the compression state always matches actual conversation content.

Provider Restriction (Paid Only)

The Kompressor makes a separate LLM call that can involve large token counts. Free-tier rate limits (Groq: 6,000–12,000 TPM; Hugging Face: variable) are insufficient for reliable compression of real-world conversations. Only paid providers are offered:

Provider	Available Compression Models
DeepSeek	`deepseek-flash`, `deepseek-v4-pro`
OpenAI	`gpt-4o-mini`, `gpt-4.1-nano`, `gpt-4o`, `gpt-4.1`, `gpt-4.1-mini`
Google	`gemini-2.5-flash`, `gemini-2.5-flash-lite`

Recommended default: DeepSeek + `deepseek-flash` — no rate limits, lowest cost per token, most reliable results.

Result Files

Each compression round is saved to disk for review:

```
/var/www/deepseek-chat/kompressor/kompressor_YYYYMMDD_HHMMSS.txt
```

Quota & Limit Banners

Red Banner — **"Credit must be renewed"** (paid providers) · Triggered by exhausted credit on a paid API. · **DeepSeek**: HTTP 402 response. · **OpenAI**: HTTP 429 + `insufficient_quota` or `credit_balance_exhausted` in JSON response body. Spend/usage limits (`organization_spend_limit_exceeded`, `project_spend_limit_exceeded`, `organization_usage_limit_exceeded`) use the same banner with the text "Spending or usage limit reached". · Displayed as a fixed-position element at the top of the viewport until manually closed (× button).

Blue Banner — **"Daily limit reached"** (free-tier providers) · Triggered by exhausted daily quota on a free API. · **Google Gemini**: HTTP 429 + daily quota keywords in response body. · **GroqCloud**: HTTP 429. · **Hugging Face**: HTTP 429. · Same fixed-position display with × close button.

Context Window Exceeded Handling

When the API returns HTTP 400 with context-related keywords in the response body, an **interactive box** appears directly in the chat instead of a generic error message:

- **Blue-bordered box**: *"The maximum chat size of the current LLM has been reached."*
- **Green button** — **"Start new chat with current context"** (Option C):
- Current session is auto-saved.
- The last compression summary (if available) is combined with all subsequent non-compressed messages as plain text.
- A new session starts with this combined context preloaded as a file attachment — the conversation continues seamlessly with full context carry-over.
- **Blue button** — **"Start new chat without context"** (clean restart):
- Current session is auto-saved.
- New session starts with an empty context.

This enables **chained conversations** across multiple sessions — theoretically unlimited in total length.

All five CGI proxy scripts detect context overflow by checking the HTTP status code and keyword-matching the API error body, returning `error_type: "context_exceeded"` to the client.

API Proxy Documentation Headers

Each of the five CGI proxy scripts (`openai-api.py`, `deepseek-api.py`, `google-api.py`, `hugging-api.py`, `groq-api.py`) contains a structured documentation block directly after the encoding declaration:

- **Import/update date** — when the file was last updated
- **Supported models** — version, context/output token limits, capabilities (text/images/audio/video), free/paid assignment
- **Source link** — official API documentation URL with date

This ensures all model specifications are traceable directly in the source code without consulting external documentation.

Download Mode

For very long AI responses (e.g. summarizing an entire novel, several million characters), rendering the full text token-by-token in the DOM as it streams in becomes visibly slow on modest hardware. **Download Mode** is an optional toggle button next to DeepThink:

- While streaming, instead of the growing full text, only a progress notice with a running character count is shown (*"Generating response… (X characters so far)"*).
- Once streaming completes, a completion notice replaces it (*"Response complete (X characters). Use the download button below this message to save it."*) instead of the full text.
- The full response text is unaffected internally — it is still built completely in memory and stored via `addMessageToHistory()` exactly as before, so context continuation, session saving, and the existing per-message download button (TXT/MD/RTF/PDF) all work exactly as they did without Download Mode.
- Download Mode is independent of Chat/DeepThink mode — the two toggles combine orthogonally.

Responsive Design

The interface was originally built desktop-first. As of 14 September 2026, it also adapts cleanly to smartphone and tablet screens:

- **Viewport meta tag** — the single most impactful fix: without `<meta name="viewport" content="width=device-width, initial-scale=1">`, mobile browsers render the page at a virtual desktop width (~980px) and zoom out, forcing pinch-to-zoom navigation. Its absence was the main cause of the interface appearing tiny on phones.
- **Two targeted media query breakpoints** (`max-width: 768px` and `max-width: 480px`) reduce body padding, slightly shrink the header title, and raise button height to the recommended 44px minimum touch target — on top of the pre-existing fluid flexbox layout (button row already wrapped, message bubbles already used `max-width`, the input field was already `width: 100%`).

- No other structural changes were needed — the existing CSS architecture (all styling lives in a single inline `<style>` block in `index.html`, no separate CSS file) was already close to mobile-friendly.

DeepSeek V4 Migration

Background

On **24 April 2026**, DeepSeek released the **DeepSeek V4 Preview** — a new generation of MoE (Mixture of Experts) language models with dramatically expanded capabilities. The two new models replace `deepseek-chat` (V3) and `deepseek-reasoner` (R1).

New Models

Model	Parameters	Active	Context	Max Output	Thinking Mode
`deepseek-v4-flash`	284B total	13B	1,048,576 tokens	8,192 tokens	Yes (Thinking + Non-Thinking)
`deepseek-v4-pro`	1.6T total	49B	1,048,576 tokens	32,768 tokens	Yes (Thinking + Non-Thinking)

Architecture Improvements (V4 vs. V3)

- **Hybrid Attention**: V4 combines Compressed Sparse Attention (CSA) and Heavily Compressed Attention (HCA) — enabling 1M-token context with only 27% of the single-token inference FLOPs of V3.2 and only 10% of the KV cache.
- **Manifold-Constrained Hyper-Connections (mHC)**: Strengthens residual connections for more stable signal propagation across layers.
- **Three reasoning effort modes**: Non-think (fast), Think High (logical analysis), Think Max (full reasoning extent) — accessible via API parameters.

Deprecation Timeline

Date	Event
24 April 2026	V4 Preview released. `deepseek-chat` and `deepseek-reasoner` begin routing to `deepseek-v4-flash`.
24 July 2026	**`deepseek-chat` and `deepseek-reasoner` fully retired and inaccessible.**

Changes Made in This Project (11 May 2026)

`index.html`: · `MODEL_CONFIG`: `deepseek-chat` (128k tokens) → `deepseek-v4-flash` (1,048,576 tokens); `deepseek-reasoner` (64k tokens) → `deepseek-v4-pro` (1,048,576 tokens) · `MODEL_CONSOLIDATION` updated to `deepseek-v4-flash` and `deepseek-v4-pro` · `DEFAULT_MODELS`, `COMPRESSOR_MODELS.deepseek` updated to V4 names · Model dropdowns (model select + compressor model select): V4 options · DeepThink logic (8 occurrences): both modes read `settings.selectedModel` (falling back to `deepseek-v4-flash`) — see the 19 July 2026 update below for a bug affecting this logic that has since been fixed · Default settings: `selectedModel` and `compressorModel` default to `deepseek-v4-flash` · Frontend error handler fixed: `response.json()` body consumption no longer causes empty error messages

`deepseek-api.py`: · Header comment updated to V4 models with correct context/output sizes · Default model fallback: `'deepseek-chat'` → `'deepseek-v4-flash'` · Deprecation notice added to header

`deepseek-models.py`: No changes needed — fetches model list live from the DeepSeek API. Already returns `deepseek-v4-flash` and `deepseek-v4-pro` correctly.

API Compatibility

The DeepSeek V4 API uses the same base URL and OpenAI-compatible format as V3. No structural changes to `deepseek-api.py` were required — only the model names needed updating.

---

19 July 2026 Maintenance & Feature Update

A full-day maintenance and feature session covering a critical model-selection bug, several accumulated documentation/model-list inconsistencies across all five providers, and the completion of a previously non-functional vision (image) pipeline. Documented in changelog entries 87–89.

1. Critical Bug: `deepseek-v4-pro` Was Never Reachable

Symptom: Selecting `deepseek-v4-pro` in the model dropdown had no effect — every request, regardless of dropdown selection or DeepThink mode, was sent using `deepseek-v4-flash`.

Root cause: Eight occurrences of the same copy-paste error across `index.html`, each resolving to:

```
currentMode === 'deepthink' ? 'deepseek-v4-flash' : 'deepseek-v4-flash'
```

Both branches of the ternary returned the identical string, meaning `settings.selectedModel` was never consulted for the DeepSeek provider — unlike every other provider (Google, OpenAI, Groq, Hugging Face), which correctly used `settings.selectedModel || fallback`.

Fix: All eight occurrences (spanning `sendMessage()`, `handleResponse()`, the context/token estimation functions, and UI indicator updates) now read `settings.selectedModel || 'deepseek-v4-flash'`, consistent with the pattern used elsewhere.

Related fix — stale context header: `saveSettings()` (the "Apply Settings" button in the LLM Settings panel) now explicitly calls `updateContextDisplay()` as its final action, guaranteeing the `Context: … (model)` header reflects the just-saved model selection immediately rather than only on the next context recalculation.

Related fix — misleading model header: The `Model: …` header previously showed, for DeepSeek only, the entire list of models reported by `deepseek-models.py` (e.g. `Model: deepseek-v4-flash, deepseek-v4-pro`) regardless of which model was actually selected — inconsistent with every other provider, which correctly showed only the active model. `updateApiStatus()`, the general UI update routine, and `forceHeaderUpdate()` were all corrected to display `settings.selectedModel` uniformly across all providers.

2. Diagnostic Logging: Model Name in Server Log

`deepseek-api.py`'s `log_to_file()` and `send_error()` functions gained an optional `model` parameter. Every log line now includes `| model=<name>` from the point the request body has been parsed onward — for both successful requests and error responses — enabling server-side verification of which model actually received a given request, independent of (and more reliable than) client-side UI state or the model's own self-reported identity (see DeepSeek Model Self-Reporting).

3. Dead Model Cleanup

Several models referenced in code comments and selection lists were no longer reachable:

Provider	Removed model	Reason
Google	`gemini-2.0-flash`	Shut down 1 June 2026; was also the CO2 script's default fallback model
Google	`gemini-1.5-pro`	Retired prior to this cleanup
HuggingFace	`mistralai/Mixtral-8x7B-Instruct-v0.1`	No longer deployed by any Inference Provider on the HF router
GroqCloud (docs only)	`mixtral-8x7b-32768`	Deprecated by Groq since 20 March 2025 (documentation was stale; the live model arrays were already correct)
GroqCloud (docs only)	`gemma2-9b-it`	Deprecated by Groq since 8 October 2025 (documentation only)
OpenAI	`gpt-4o-mini`, `gpt-5.2-chat-latest`	No longer on OpenAI's current model/pricing listing (superseded by the GPT-5.4/5.5/5.6 family)
Google	`gemini-2.5-pro`	Blocked for new accounts (HTTP 404, 12-10-2026); replaced by `gemini-3.1-pro-preview`

Provider	Removed model	Reason
HuggingFace	mistralai/Mistral-7B-Instruct-v0.3, microsoft/Phi-3.5-mini-instruct, meta-llama/Meta-Llama-3.1-70B-Instruct, meta-llama/Meta-Llama-3.1-405B-Instruct	Not a chat model / supported by no enabled provider / no longer existing (HTTP 400, 12.10.2026)
GroqCloud	llama-3.1-70b-versatile, llama-3.1-8b-instant, meta-llama/llama-4-scout-17b-16e-instruct, qwen/qwen3-32b, moonshotai/kimi-k2-instruct-0905	No longer offered by Groq (HTTP 404, 12.10.2026)
OpenAI (audio only)	[illegible]	Reject `input_audio` (HTTP 400, 12.10.2026); removed from `AUDIO_CAPABLE_MODELS` only

Removed from [illegible], [illegible], [illegible], `MODEL_LIMITS`, [illegible], [illegible], and [illegible] as applicable. `google_api.py`'s default fallback model was changed from [illegible] to [illegible]. A leftover orphaned `MODEL_LIMITS` entry for the removed Mistral model (present after it had already been removed from all selection lists) was also cleaned up.

Additionally, `MODEL_LIMITS` was missing an entry entirely for [illegible] even despite it being listed in [illegible] — it silently fell back to DeepSeek's output-token limit. A correct entry (131,072 context / 8,192 output) was added.

4. OpenAI Lineup Updated to GPT-5.5 / GPT-5.6

OpenAI released **GPT-5.5** (23 April 2026) and the **GPT-5.6 family — Sol, Terra, Luna** (9 July 2026, the current flagship generation) since this project's model configuration was last updated (10 March 2026). See OpenAI Integration for the updated model lists.

A second, independently discovered issue: `MODEL_LIMITS` had **zero entries for any OpenAI model** prior to this update. Every OpenAI request — regardless of which model was selected — silently used DeepSeek Flash's limits (8,192 max output tokens), underutilizing models like [illegible] (32,768 real max output) or the GPT-5.6 family (128,000 real max output). Eight correct entries were added, with context/output values sourced from OpenAI's current API documentation.

5. Transparent API Error Messages

Symptom: API errors from DeepSeek, OpenAI, Groq, and Hugging Face displayed as a bare [illegible] with no further detail, making diagnosis impossible from the UI alone.

Root cause: In three separate error-handling branches within [illegible], the parsed backend error response ([illegible]) was discarded once it had been checked against the two specific known error types ([illegible]). If the actual error was neither of those — e.g. an unrecognized parameter, as encountered in point 6 below — the thrown error message was hardcoded to an empty string.

Fix: All three branches now extract [illegible] and pass it through to the displayed error message, surfacing the real, provider-supplied diagnostic text.

6. `max_tokens` → `max_completion_tokens` for OpenAI

Discovered as a direct consequence of fix 5 above: once real error text became visible, the first live test with [illegible] returned:

```
Unsupported parameter: 'max_tokens' is not supported with this model. Use 'max_completion_tokens' instead.
```

OpenAI's current-generation models (GPT-5.x and reasoning-capable models generally) reject the `max_tokens` parameter in the Chat Completions API. `openai_api.py` now sends `max_completion_tokens` instead — a parameter accepted by both legacy models (GPT-4o, GPT-4.1) and the current GPT-5.x lineup, making it safe to use unconditionally rather than branching per model. Verified live against both [illegible] and [illegible].

7. Vision Pipeline Completed

See the dedicated Vision (Image) Support section above for the full description. In summary: [illegible] was populated correctly per provider (previously only DeepSeek had entries), all [illegible] was corrected to check the actually-selected model instead of an unrelated DeepSeek-only array, and the client/server plumbing to actually transmit image bytes to Google Gemini and OpenAI was implemented for the first time — images had previously been accepted by the UI but never reached any model.

8. `deploy.sh`: MD5 Checksum Verification

`deploy.sh` now prints the MD5 checksum of every file it copies into the production directory, immediately after the copy/chown/chmod steps and before the Apache reload:

```
=== MD5-Summen der deployten Dateien (Produktion) ===
[illegible]  /var/www/deepseek-chat/index.html
[illegible]  /var/www/deepseek-chat/cgi-bin/openai_api.py
...
```

This removes the need for a separate manual `md5sum` step after every deploy to confirm the production file matches the intended commit — a step that had previously caught a stale-deploy issue during this same session (a production `deploy.sh` invocation was running an out-of-date cached copy of itself, since the script does not deploy itself; see the note in Deploy Scripts).

Files Changed

`index.html`, `deepseek_api.py`, `google_api.py`, `groq_api.py`, `hugging_api.py`, `openai_api.py`, `shell-scripts/deploy.sh`

---

28 August 2026 Fix: Explicit DeepSeek Thinking Mode Control

Symptom: No visible bug — discovered while researching a self-reported model version question. Chat mode and DeepThink mode had always been documented (see DeepThink Mode) as differing only in the system prompt, never in an actual API-level reasoning parameter.

Root cause: As of the DeepSeek V4 API, Thinking Mode is enabled by default at [illegible] reasoning effort for every request unless explicitly disabled via [illegible] in the request body — this is no longer tied to the model name (unlike the legacy [illegible] split). `deepseek_api.py` never set this parameter, so **every** request to [illegible] — including plain Chat mode — was silently reasoning at full effort, incurring the associated reasoning-token cost and latency the Chat mode is meant to avoid.

Fix: The frontend now sends an explicit [illegible] boolean ([illegible] only when DeepThink mode is active) alongside the existing DeepSeek request payload. `deepseek_api.py` forwards this as [illegible] when true, or [illegible] when false, making the Chat/DeepThink distinction real at the API level instead of system-prompt-only.

Verification: DeepSeek's own API documentation ([illegible]) was consulted directly to confirm the default-enabled behavior and the exact control parameters before implementing the fix.

Files Changed

[illegible]

Security Hardening (26 July – 15 September 2026)

A structured, staged security review of the entire application — prompted by a second LLM's independent audit of the codebase — closed a series of real vulnerabilities across the frontend and all CGI backend scripts. Each stage was implemented, deployed, and verified (MD5 checksum against the source repo, plus a live exploit attempt test) before moving to the next.

Stage 1 — Stored XSS and Path Traversal

Three stored cross-site-scripting points were found and fixed in `index.html`, all sharing the same root cause: user-controlled text rendered via `innerHTML` instead of `textContent`:

- **Chat messages** — [illegible] rendered the user's own message via `innerHTML`. A message like [illegible] executed arbitrary JavaScript for anyone viewing the chat history (AI responses were already safely rendered via `textContent`).
- **Session list preview** — [illegible] inserted [illegible] and [illegible] (the first user message of a saved session) directly into an `innerHTML` template.
- **PDF upload filename** — the upload progress indicator inserted [illegible] via `innerHTML`; a maliciously named PDF ([illegible]) sent to a victim would execute on upload.

All three were fixed by switching to `textContent`, or by building the static HTML skeleton via `innerHTML` and filling in only the dynamic, untrusted parts afterward via [illegible].

Separately, [illegible], [illegible] and [illegible] validated only the date/time portion of the session ID, never the random suffix — a session ID such as [illegible] passed the old check and allowed arbitrary file read/write/delete on the server. Fixed with two independent layers: a strict [illegible] regex, plus a [illegible] helper that resolves the final path via [illegible] and rejects it if it escapes the sessions directory — either layer alone stops the attack.

Stage 2 — CORS Wildcard and Request Size Limits

Every CGI endpoint sent `Access-Control-Allow-Origin: *`, meaning any website — including one opened in another tab on the same VPN-connected device — could make cross-site requests against the API and read the responses. Replaced with a fixed allowed origin across all 16 affected scripts. Two export scripts ([illegible], [illegible]) already omitted the CORS header entirely, which is the strictest possible default and was left untouched.

No endpoint enforced a request size limit, an available denial-of-service vector. Added a [illegible] check (20 MB for endpoints that may carry embedded images, 64 KB for the deliberately tiny [illegible] payload) with an HTTP 413 response on violation, verified against both an oversized and a normal-sized request via a live simulated CGI call.

Stage 3 — Log Exposure and Host Header Injection

[illegible] returned the entire log file unfiltered; limited to the last 300 lines via [illegible] for efficient tail reading without loading the whole file into memory.

The Apache HTTP→HTTPS redirect used the client-supplied [illegible] header unvalidated in the target URL — a forged [illegible] header could redirect visitors to an attacker-controlled domain. Replaced with the server's own fixed address. (This particular vhost, [illegible], later turned out to be disabled in production via [illegible] — the fix is correct and in place for if it's ever re-enabled, but doesn't currently execute; documented in the design manifest rather than presented as an active production fix.)

Stage 4 — Deliberately Not Implemented: Endpoint Authentication

No authentication layer exists in front of [illegible]. Given the server's actual threat model — unreachable from the public internet, accessible only via SSH or the WireGuard VPN gateway (see Architecture), single operator — the added complexity of Basic Auth or a shared-secret header was judged disproportionate to the marginal security gain. This is a considered decision, not an oversight, and is recorded as such in the project's design manifest so a future audit (human or AI) does not need to re-flag it.

Stage 5 — Documentation Gap and Deferred Modularization

[illegible] existed on the production server but was never committed to this repository — fixed by adding it. A related architectural observation — [illegible] has grown to roughly 4,700 lines with all JavaScript in one inline [illegible] block — was raised and deliberately left as-is; splitting it into modules is a larger refactor with no functional benefit, tracked as an open, low-priority item rather than acted on.

Residual XSS Follow-Up (14 September 2026)

A second pass caught spots the first round missed: the magic-byte detection error path and four [illegible] calls in the upload handler still inserted [illegible] via [illegible], and the session-list load error handler inserted [illegible] the same way. Fixed with a new [illegible] helper (the standard [illegible] browser trick) wrapped around every remaining untrusted value. Verified live by uploading a file literally named [illegible] — both on a parse failure and on magic-byte executable detection, the name rendered as inert text.

All fixes are recorded in changelog entries 91–100, with MD5 checksums for every deployed file.

HTTPS Migration: From Self-Signed to Let's Encrypt

Through 14 September 2026, the client used a self-signed certificate generated with [illegible]. Functionally valid TLS, but every new device showed a "Not secure" warning until its browser was taught to trust the [illegible] root CA manually — workable on a Linux workstation, painful on Android (no straightforward CA import path for a browser without extra apps).

The Fix

A real, publicly trusted certificate via **Let's Encrypt**, issued through [illegible] using a **DNS-01 challenge** against the DynDNS provider Dynu ([illegible]). DNS-01 was essential here: it proves domain ownership purely via a temporary TXT record, without ever needing the server to be reachable from the public internet — the actual chat traffic stays exactly as unreachable as before.

A Naming Trap, Caught Before Deployment

The obvious choice — reusing the existing [illegible] hostname, already registered for the WireGuard endpoint — was tested and then deliberately abandoned. That hostname is resolved by every WireGuard client before the VPN tunnel exists, to find the server at all; overriding its DNS answer for browser traffic once inside the tunnel would risk the client resolving the wrong address during a reconnect and being unable to re-establish the tunnel at all. A second, dedicated hostname — [illegible] — was registered instead, with its public DNS A record pointing directly at the server's private VPN address. This makes the private IP visible to anyone who looks it up publicly, which was judged acceptable: the server remains unreachable from outside the VPN/SSH path regardless of what its address is.

Result

- Certificate auto-renews via the cron job [illegible] installs itself.
- No CA import needed on any client device, ever again — including Android, without root.
- The original WireGuard endpoint hostname is untouched and still resolves publicly as before.

Side Effects Found Along the Way

Validating this change surfaced three unrelated bugs in the underlying VPN gateway infrastructure (tracked in the separate [illegible] repository, mentioned here only because they were found while working on this project's HTTPS setup): a duplicated YAML key in the shared DNS resolver's config silently dropped two of three intended upstream DNS-over-TLS providers down to just one; a leftover test instance of [illegible] was squatting on the same port the intended resolver needed, causing intermittent resolution failures; and an overly permissive firewall rule forwarded outbound DNS-over-TLS (port 853) through an unrelated VPN egress path, letting a Chromium-based browser's automatic secure-DNS probing bypass the intended resolver entirely. All three are fixed in that repository; none affect this application's own code.

---

17 September 2026: Model Display Labels & Cleanup

Removed: `deepseek-v4-flash-vision-exp`

DeepSeek's original experimental vision model was removed from the model selection entirely. Since 10 September 2026, DeepSeek itself routes this model name server-side to [illegible] (V4.1 Flash) anyway — keeping it as a separate, selectable option was purely redundant. The generic, model-agnostic image-upload code path in [illegible] needed no change; it continues to work unchanged for [illegible].

Added: Readable Model Labels with Release Dates

Model dropdowns previously showed the raw API model ID directly (e.g. [illegible], [illegible]) — functional, but not obviously informative about which generation or how current a model is. A new [illegible] map plus a [illegible] helper decouples the value actually sent to the provider's API from the text shown in the dropdown: labels read [illegible], deliberately without any language-specific word (an initial version used the German "Stand", which broke for English/Spanish UI users — removed in favor of a bare date, which needs no [illegible] translation entry at all). Covers all 23 models across all five providers; any future model without an entry falls back cleanly to its raw ID.

Every release date was individually verified against public sources rather than assumed — several of the dates already documented in the [illegible] header comments turned out to be the file's own edit date rather than the model's actual historical release date (most visibly for [illegible], [illegible], and [illegible], which are considerably older than the proxy script that documents them). Corrected in [illegible], [illegible], [illegible], and [illegible].

A Bug in the First Pass

The initial implementation updated only the one function that rebuilds the model dropdown when switching providers — missing four separate functions that also rebuild it, triggered when toggling a provider's Free/Paid tier (and when restoring saved settings on page load). Since DeepSeek has no tier toggle, the gap went unnoticed there and only surfaced once OpenAI and Google were labeled. Fixed in all four functions; every subsequent provider (Hugging Face, GroqCloud) was then checked with an exhaustive search across the entire file for every place that ever constructs a model [illegible], not just the first one found.

All changes recorded in changelog entry 106.

The Helper Script `repo2text.sh`

This Bash script was specifically developed to **export the entire source code of a GitHub repository as a single text file** — ideal for passing the complete project context to an AI assistant in a single upload.

How it works: - Clones the repository with `[illegible]`. - Analyzes all text files (MIME type check + `[illegible]`) and writes them sequentially with unique delimiters into an output file. - Uses `[illegible]` to deduplicate file paths before processing — prevents duplicate file entries in the output. - Uses a unique delimiter format (`[illegible]`) that cannot appear in source code. - Explicitly respects `[illegible]` and `[illegible]`. - Supports TXT, JSON, and Markdown output formats. - Creates a ZIP archive of the export file. - Includes metadata: commit hash, branch, export timestamp.

Special options: - `[illegible]` Use only filenames without directory paths. - `[illegible]` Export only a specific subdirectory. - `[illegible]` Compute and include MD5 checksum for each file. - Intelligent remote URL detection when run inside an existing Git repository. - Both `[illegible]` (Linux) and `[illegible]` (macOS) are supported.

Usage examples:

```
# Simple export (interactive URL prompt)
./repo2text.sh

# Export with URL as Markdown format
./repo2text.sh -f md https://github.com/debian-professional/multi-llm-chat.git

# Export only the 'shell-scripts/' directory with flat structure
./repo2text.sh --flat -o shell-scripts https://github.com/debian-professional/multi-llm-chat.git

# Export with MD5 checksums
./repo2text.sh --md5 https://github.com/debian-professional/multi-llm-chat.git
```

`repo2text` is also available as a standalone project: github.com/debian-professional/repo2text

---

Security Architecture in Detail

Security was a top priority throughout development. All key measures:

1. API Keys — Never Exposed to the Client

- All API keys are held exclusively in Apache environment variables (`[illegible]`).
- Each CGI script retrieves its key via `[illegible]`.
- The client communicates only with local CGI proxies — never directly with external APIs.
- Even a full XSS compromise of the page cannot leak API keys.

2. Magic Byte Inspection

- The first 20 bytes of every uploaded file are checked against a comprehensive signature database covering 12 executable formats across 6 platforms.
- If a signature matches, the upload is blocked before any content is read — with a detailed error message showing the detected platform and format.
- Protection works even if malicious files are renamed (e.g. `[illegible]` → `[illegible]`).

3. Secure Session Storage

- Sessions directory: `[illegible]` — `[illegible]`, owned by `[illegible]`.
- Each session file: `[illegible]`.
- Session IDs validated server-side via regex before any file I/O — no path traversal possible.

4. Log Without Sensitive Data

- Logged: timestamps, IP addresses, HTTP methods, endpoint paths, status codes, error messages.
- **Never logged:** API keys, session contents, full message text (only 60-char feedback previews).
- OPTIONS preflight requests are filtered to prevent log flooding.

5. No Direct Client-API Communication

- All security-critical operations are server-side Python CGI.
- The client has zero knowledge of API credentials, server paths, or session storage locations.

6. Input Validation

- Files validated by extension allowlist AND magic byte inspection.
- Session IDs validated against expected format regex server-side.
- Clipboard paste filtered to block file paths before they reach the API.
- `[illegible]` validated before reading POST bodies in CGI scripts.

7. Transport Security

- HTTPS enforced via `[illegible]` with Apache mod_ssl.
- Plain HTTP configuration (`[illegible]`) disabled via `[illegible]`.
- Since 15 September 2026, the TLS certificate is a real, publicly trusted Let's Encrypt certificate (auto-renewing via `[illegible]`), replacing an earlier self-signed `[illegible]` certificate — see HTTPS Migration for the full story, including why the certificate's domain is deliberately separate from the WireGuard VPN endpoint hostname.

Deployment & Usage

Prerequisites

- Debian-based Linux (or any Linux with Apache 2.4, Python 3.9+, Bash)
- Apache modules: `[illegible]`
- Python packages: `[illegible]` (for PDF export)
- For `[illegible]`: `[illegible]`
- Valid API key for at least one supported provider

Installation

1. Clone the repository (as user source):

```
git clone https://github.com/[illegible]/[illegible]-llm-chat.git /home/source/[illegible]-llm-chat
```

2. Configure API keys in /etc/apache2/envvars:

```
export DEEPSEEK_API_KEY="sk-..."
export OPENAI_API_KEY="sk-proj-..."
export GOOGLE_API_KEY="AIza..."
export HF_API_KEY="hf_..."
export GROQ_API_KEY="gsk_..."
```

3. Enable Apache configuration:

```
a2ensite deepseek-chat-ssl.conf
a2dissite deepseek-chat.conf
systemctl restart apache2
```

4. Create required directories:

```
mkdir -p /var/www/deepseek-chat/sessions
chown www-data:www-data /var/www/deepseek-chat/sessions
chmod 700 /var/www/deepseek-chat/sessions
```

5. Deploy (as root):

```
./deploy.sh source
```

6. Install helper scripts:

```
./install.sh  # copies deploy.sh and sync-back.sh to production directory
```

Configuration

Model configuration (MODEL_CONFIG in [illegible]) — single point of truth for all model limits, current as of 17.09.2026:

```
const MODEL_CONFIG = {
[illegible]
};
[illegible]
```

Since 17 September 2026, the visible label for every model in the dropdowns above is looked up separately via [illegible] — see Model Display Labels & Cleanup. The arrays above still define which raw IDs are valid and sent to each provider's API; only the on-screen text differs.

Language configuration (language.xml): Add a [illegible] block to activate the custom language slot. Set [illegible] for languages with formal/informal distinction.

Deploy Scripts

Script	Function
deploy.sh [illegible]	Copies files from [illegible] to /var/www/deepseek-chat/, sets ownership and permissions, prints MD5 checksums of every copied file, reloads Apache
sync-back.sh [illegible]	Copies changed files from production back to the source repo
install.sh	Installs deploy.sh and sync-back.sh in the production directory
tag-release.sh	Creates a Git tag with auto-incremented version number and pushes it. Runs git fetch --tags first to avoid conflicts with existing remote tags.
api-test.py	Tests all providers/models with the proxies' real parameters ([illegible]). Runs only on the server (Apache2 production directory and valid keys required, otherwise abort). Not deployed to production.
web-test.py	Browser test of the web page with simulated provider answers ([illegible]). Not deployed to production.

Important — deploy.sh does not deploy itself. The script's own cp list covers [illegible] — it never copies itself. After changing deploy.sh in the source repo and running git pull on the server, the production copy at /var/www/deepseek-chat/deploy.sh remains the **old** version until it is copied over manually:

```
cp [illegible]/shell-scripts/deploy.sh /var/www/deepseek-chat/deploy.sh
chmod +x /var/www/deepseek-chat/deploy.sh
```

This caught out a live deployment during the 19 July 2026 session — sudo deploy.sh kept producing the pre-MD5 checksum output even after git pull succeeded, because the invoked script was still the stale production copy.

Project Structure

```
[illegible]
```

Model Configuration

The MODEL_CONFIG object in [illegible] is the **single point of truth** for all model-specific limits across all five providers. All features that depend on model limits — context utilization display, dynamic upload limits, context exceeded detection, Kompressor thresholds — read from this single object. A model missing from MODEL_CONFIG silently falls back to DeepSeek Flash's limits rather than failing loudly — this previously affected every OpenAI model until it was corrected on 19 July 2026 (see 19 July 2026 Maintenance & Feature Update), so any newly added model should be checked against this table explicitly rather than assumed to work.

Updating model configuration: When a provider updates their models (new model, changed context limits, deprecated model), only the MODEL_CONFIG block in [illegible] needs to be updated. No other files require changes unless the model name is also used in the provider model lists ([illegible], etc.), in [illegible], or in [illegible].

Sources: OpenAI API Docs, DeepSeek API Docs, Google Gemini Docs, Hugging Face Inference Providers, GroqCloud Docs. OpenAI and DeepSeek pricing/context entries re-verified 14.09.2026; Google/Hugging Face/GroqCloud pricing/context entries as of 19.07.2026 (not re-checked in the 14.09.2026 update). Individual model release dates for all 23 models across all five providers were independently verified on 17.09.2026 for the model display label feature (see Model Display Labels & Cleanup).

Design Manifest

The project includes a [illegible] file documenting all design decisions, naming conventions, and development rules. Key rules:

- **All buttons:** Pill-style exclusively (border-radius: 20px, height: 36px). Square buttons are forbidden.
- **Button colors:** Blue ([illegible]) for actions, dark-to-blue toggle for modes, red ([illegible]) for destruction, green ([illegible]) for constructive operations.
- **Settings:** Toggle switches only — no radio buttons, no checkboxes anywhere.
- **No emojis** in buttons or labels (exception: the DeepThink icon ✦).
- **No PHP** — exclusively JavaScript (client) and Python 3 (server).
- **No external JavaScript frameworks** — no Node.js, no React, no Vue, no jQuery.
- **Formatting preservation:** Existing indentation and formatting in `[illegible]` must never be changed by automated tools.
- **`[illegible]` must be kept current:** (Manifest rule 3.1): Whenever a model gains or loses audio support, the constant must be updated immediately.
- **Provider banners required** (Manifest rule 3.1): When adding a new LLM provider, the appropriate quota/limit banner must be implemented in both the CGI script and the client.
- The manifest is a **separate file** and must never be embedded in `[illegible]`.

Known Limitations & Technical Notes

"Lost in the Middle" — A Known AI Limitation

All current language models tend to reliably recall content at the **beginning and end** of a long context, while content **in the middle** is sometimes overlooked or hallucinated. (Liu et al., 2023: *"Lost in the Middle: How Language Models Use Long Contexts"*)

Practical impact: - A repository export of this project is approximately 700,000 characters ≈ ~175,000 tokens. - DeepSeek V4 models (`[illegible]`, `[illegible]`) have a 1M token context window — the full repository export fits comfortably. - Google Gemini with 1–2M token context handles the export without issues. - OpenAI models with 128k context (e.g. `[illegible]`) **cannot** load the full export — the client will block the upload with a clear error message. - **Recommendation:** Even with models that technically fit the export, upload only the relevant files for the current task to maximize the model's effective attention.

GitHub Raw URL Caching

After `git push`, the new version is **not immediately available** via `[illegible]` URLs — GitHub caches these for up to 10 minutes. This is expected behavior and cannot be circumvented. Files are correctly stored on GitHub as soon as `git push` reports success.

Nano and Unicode — Critical Warning

Never edit files containing Unicode escape sequences (such as the umlaut placeholder functions) using `nano` or by copy-pasting into a terminal emulator.

Nano corrupts `[illegible]` sequences to multi-byte garbage ([illegible]), which breaks JavaScript parsing silently.

The only safe workflow: 1. Edit files locally in a proper editor (VS Code, gedit, kate). 2. `git add` / `git commit` / `git push` from the local machine. 3. On the server: `git pull` (in the source repo as user `[illegible]`). 4. As root: `[illegible]`.

Linux/X11/Firefox Paste Behavior

On Linux with X11 and Firefox, `[illegible]` in paste event handlers does not reliably block browser-native paste behavior for content originating from file managers. The implemented workaround (allow the paste, check input content in `[illegible]`, clear and alert if file paths detected) is the only reliable solution for this platform-specific limitation.

Context Overflow Detection Edge Cases

Context overflow detection in all five CGI scripts uses HTTP status code analysis combined with keyword matching in the API error response body. The keyword set is broad enough to cover standard API error messages. However, edge cases with unusual error messages from provider infrastructure changes may not be caught and would fall back to a generic error display.

DeepSeek Model Self-Reporting

DeepSeek V4 models may report inaccurate self-knowledge when asked about their context window size or version — they respond based on their training data, not their actual API configuration. The actual deployed model (`[illegible]` or `[illegible]`) can be verified via:

```
[illegible] && curl -s https://api.deepseek.com/v1/models \
  -H "Authorization: Bearer $DEEPSEEK_API_KEY"
```

Dependencies

Component	Purpose	Installation
Apache 2.4	Web server, CGI, SSL	`apt install apache2`
Python 3.8+	All server-side CGI scripts	`apt install python3`
reportlab	Server-side PDF export	`pip3 install reportlab --break-system-packages`
PDF.js 3.11.174	Client-side PDF text extraction	CDN (automatic fallback to secondary CDN)
jq	JSON processing in `[illegible]`	`apt install jq`
pv	Progress display in `[illegible]`	`apt install pv`
git	Version management	`apt install git`
zip	Archive creation in `[illegible]`	`apt install zip`

No exotic frameworks — all dependencies are standard packages in a Debian environment or well-established CDN libraries.

Conclusion / Why This Project Stands Out

This project demonstrates professional-level web development in a minimalist, security-first approach — without unnecessary overhead, but with the highest standards for security, correctness, and user-friendliness.

Architecture: - Clean separation of client (pure HTML/JS) and server (Python CGI) with no blurring of responsibilities. - API keys never exposed — even a full XSS compromise cannot leak them. - Single-file client (`[illegible]`) that is entirely self-contained yet highly modular internally. - Zero build pipeline — the development environment is identical to production.

User experience: - Streaming responses with sub-second first-token latency. - Unique flexible context management — delete any message and all following ones. - Intelligent clipboard handling for text, images, and file path protection. - Audio recording directly in the browser for Gemini (all models). - Kompressor — automatic context compression enabling indefinitely long conversations. - Context exceeded handling — interactive in-chat box with smart context carry-over (Option C). - Quota banners — clear, persistent visual feedback for exhausted credit or daily limits. - Copy to clipboard — entire chat exported client-side with a single click. - Multi-language support with address form distinction, loaded from external XML.

Engineering: - Magic byte inspection detecting executables regardless of filename extension — 12 signatures across 4 platforms. - Umlaut placeholder system solving a fundamental DeepSeek API limitation for German text. - Forward-compatible model capability map — adding a new model requires a single config entry (with `[illegible]` and `[illegible]` kept explicitly in sync, a gap that had allowed OpenAI's entire model lineup to silently run on the wrong output-token limit until 19 July 2026). - End-to-end vision pipeline for Google Gemini and OpenAI — base64 image data flows from browser upload/paste through to native `[illegible]` API payloads, gated by a per-model capability map rather than a hardcoded assumption. - Precise compressor summary discard: summary invalidated when context drops below the last triggered threshold after manual deletion. - Dynamic upload limit: 75% of the active model's context window in characters — automatically scales from 384k chars (`[illegible]`) to ~3.1M chars (`[illegible]`, `[illegible]`, the GPT-5.4 family). - Deploy verification built into the deployment pipeline — `[illegible]` prints MD5 checksums of every copied file, catching stale/mismatched deploys immediately rather than discovering them through unexplained runtime behavior. - Complete audit trail via Git, detailed 106-entry changelog, and design manifest.

DeepSeek V4.1 ready — `[illegible]` (native image understanding) joins `[illegible]` (1M token context windows; `[illegible]` is a retired alias and no longer selectable since 12.10.2026), well ahead of the (already passed) 24 July 2026 legacy model retirement deadline. The interim experimental `[illegible]` was removed 17 September 2026, fully superseded.

GPT-6 Astra ready — OpenAI lineup current through the new Astra flagship (3/4 September 2026), GPT-6.1 Sol and GPT-6 Luna (September 2026) and the Sol/Terra/Luna generation (9 July 2026), with `[illegible]` used throughout for compatibility across the full model range.

Security-audited and publicly trusted — a staged security review (26 July – 15 September 2026) closed stored XSS, path traversal, CORS, and request-size gaps across the backend; the site now serves a real Let's Encrypt certificate instead of a self-signed one, with no public port exposure required.

Readable model labels — all 25 models across all five providers show a plain-language name and real release date instead of a raw API ID, added 17 September 2026.

For a professional developer, this project demonstrates: - **Security awareness** — API key protection, executable detection, secure session storage, no path traversal. - **Structured discipline** — design manifest, version tags, strict UI conventions, 106-entry changelog. - **Problem-solving depth** — X11 paste behavior, umlaut corruption, PDF binary output issues, "Lost in the Middle", context overflow chaining, and a same-day root-cause chain from a blank error message to a missing OpenAI request parameter. - **Complete documentation** — inline code comments, dedicated manifest, per-script documentation headers, three-language README.

Last updated: 17.09.2026